

1 What does the bridge from booked to posted look like?



Invented numbers, on the day's two tiny tables. Booked comes from `orders` alone and posted from `payments` alone, and each move has a list of orders behind it.

Crux. A join is done when its row count is explained: rows in, rows out, the difference named.

2 Which join answers which question?

Join	The question it answers	Rows on the tiny tables
INNER	Which orders were matched, once per payment?	6
LEFT, orders first	What happened to every order, paid or not?	7
RIGHT	What happened to every payment, matched or not?	7
FULL OUTER	What fails to match, on either side?	8

Anand asked about every booked order, so his report is a LEFT JOIN with `orders` first. The platform lead's question about every payment row starts from `payments`.

Crux. Start from the table whose every row must survive, and name its grain.

3 Why does a join double the bookings, and how do you stop it?

A two-installment order owns two payment rows, so a join writes its amount twice: Q2's 462 orders become 678 rows, and the first draft reports Rs 19,29,04,410 against Rs 9,84,00,000 booked. DISTINCT loses Rs 20,32,780, since orders share amounts.

```
WITH posted_per_order AS (  
  SELECT order_id, sum(amount) AS posted  
  FROM payments GROUP BY order_id)  
SELECT o.order_id, o.amount, pp.posted  
FROM orders o LEFT JOIN posted_per_order pp  
  ON pp.order_id = o.order_id  
WHERE o.quarter = 'Q2';
```

Crux. Bring the many side to the grain of the question before you join.

4 Where does a condition on the payments table go?

In WHERE it runs after the join: an unpaid order's NULL date fails the test, and the LEFT JOIN turns INNER without a word. In ON it decides which payments attach, and every order survives.

```
LEFT JOIN payments p  
  ON p.order_id = o.order_id  
  AND p.paid_date BETWEEN '2026-07-01'  
                        AND '2026-09-30'  
WHERE p.order_id IS NULL -- the anti-join
```

With the dates in ON the list means "not paid within Q2"; for "never paid", leave dates off payments. Avoid `NOT IN`: one NULL id in the subquery and it returns no rows at all. `NOT EXISTS` reads as Anand's sentence and stays correct.

Crux. In a LEFT JOIN, a condition on the right-hand table goes in ON.

5 How do you tell a retry from an instalment?

Pattern per order	What it is	What to do
Instalments 1 and 2	The customer paid as asked	Count both
Instalment 1, twice	The gateway retried	Count it once; list the surplus

`GROUP BY order_id HAVING count(*) > 1` flags 216 Q2 orders, every instalment order among them. Group by `order_id`, `instalment_no`, and the list's surplus, `sum(amount) - max(amount)`, must equal posted less collected.

Crux. Two payment rows are not a double payment: a retry is one order and instalment, twice.

6 Which checks let the number leave?

Tie-back check	Stops
Orders against the orders table	a dropped or repeated order
Booked against <code>orders</code> alone	a fan-out
Gap against booked less collected	a NULL gap; use <code>coalesce(collected, 0)</code>
Gap against never-paid plus paid-short	a dropped order, a wrong list
Collected plus posted twice against posted	a retry inside collected

When one fails late on reporting day, booked leaves with the open line named; collected is held.

Crux. A check is worth its power to fail: tie every figure back to one table alone.

THE WORDS ON THIS SHEET

- Grain** What one row of a table stands for, said before any join
- Anti-join** The rows of one table with no partner in the other
- coalesce** Returns its first argument that is not NULL
- Revenue bridge** Booked walked to posted in named moves, each a list of orders
- Gateway retry** The same order and instalment posted twice by the payment feed
- Tie-back check** A figure recomputed outside the report, from the source tables, and compared